

Deep Learning and Applications

Course 5

Attention and Transformers

Amaury Habrard

`amaury.habrard@univ-st-etienne.fr`

LABORATOIRE HUBERT CURIEN, UMR CNRS 5516
Université Jean Monnet Saint-Étienne

Semester 1

Today's content

- ▶ Concept of Attention
- ▶ Transformers

Credits: we use the slides/resources of F. Fleuret, K. Gimpel, T.-N. Le, D. Batra, C. Verloot, S. Wiegrefe, N. Fijalkow and The Fidle course on transformers.

https://www.cc.gatech.edu/classes/AY2020/cs7643_fall/

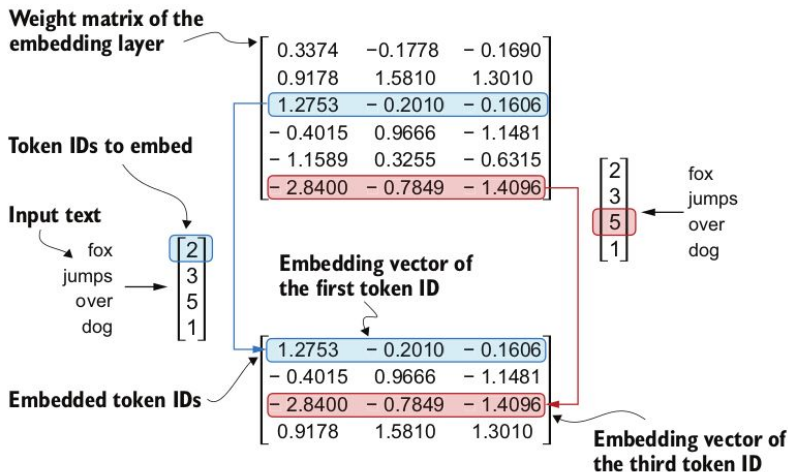
<https://fleuret.org/dlc/> Fidle course on transformers (<https://fidle.cnrs.fr>)

Attention and Transformers

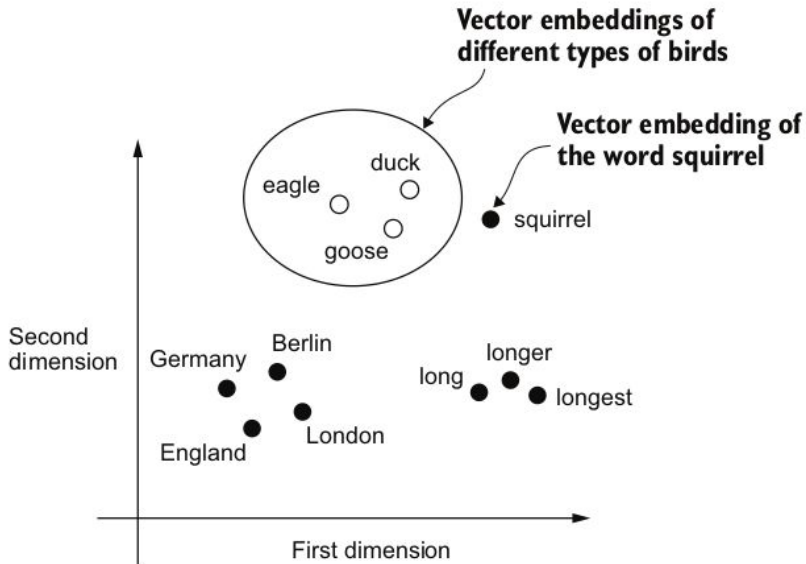
We use the slides of Francois Fleuret, Kevin Gimpel and Sarah Wiegreffe and the slides from the Fidle course on transformers
(<https://fidle.cnrs.fr>)

Some images also come from the blog of J. Alammarr:
<http://jalammar.github.io/illustrated-transformer/>

Recap on sentence embedding



Recap on Word/Token embedding



Motivations

In many models/operations, we have seen so far, such as fully connected layers, convolutions, or poolings, the contribution of a value in the input to a value in the output is mostly driven by their locations.

Some tasks require more hierarchical structures, *i.e.* in translations:

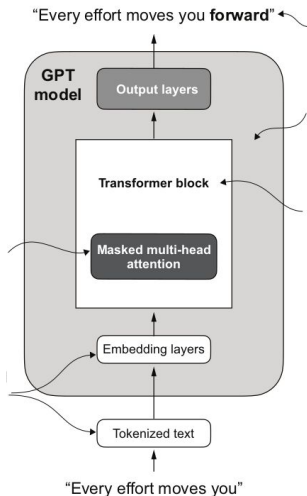
- ▶ "**An apple** that had been on the tree in the garden for weeks had finally been **picked up**."
- ▶ "**Une pomme** qui était sur l'arbre du jardin depuis des semaines avait finalement été **ramassée**."

It has motivated the development of attention-based processing to transport information from parts of the signal to other parts dynamically identified.

Attention and Transformers

Classic transformer architectures aim at generating the next word of a text, words are generated one by one.

Attention heads is a key component in a transformer.



Attention mechanism

Attention

Attention mechanisms **aggregate features with an importance score** that

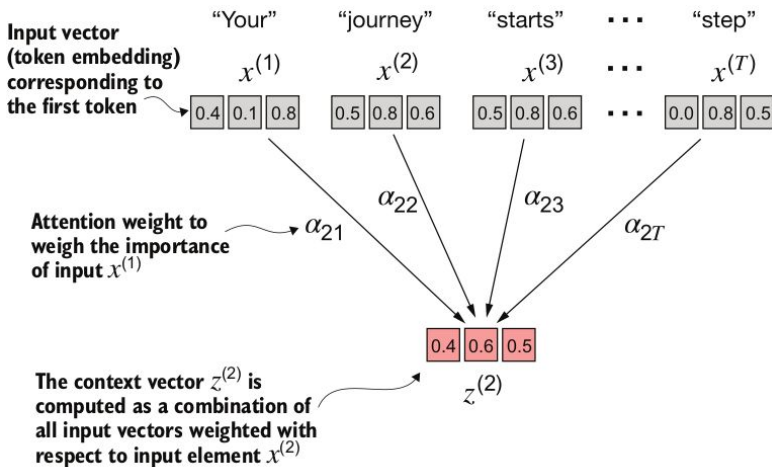
- ▶ depends on the feature themselves, not on their positions,
- ▶ relax locality constraints.

Attention mechanisms **modulate dynamically the weighting of different parts** of a signal and allow the representation and allocation of information channels to be dependent on the activations themselves.

While they were developed to equip deep-learning models with memory-like modules (Graves et al., 2014), their main use now is to provide long-term dependency for sequence-to-sequence translation (Vaswani et al., 2017).

Self-Attention

- ▶ A mechanism that propagates information between vectors
- ▶ For each input token i , it provides an embedding that captures the importance of the other tokens for the token i
- ▶ This is a Sequence to Sequence operation
 - ▶ **Input:** $x_1, x_2, \dots, x_T$
 - ▶ **Output:** $z_1, z_2, \dots, z_T$
- ▶ $z_i = \sum_{j=1}^T \alpha_{ij} x_j$
 - ▶ Simple definition: $e_{ij} = x_i \cdot x_j$
 - ▶ $\alpha_{ij} = \text{softmax}(e_{ij}) = \frac{\exp(e_{ij})}{\sum_k \exp(e_{ik})}$
- ▶ α_{ij} is a normalized attention score: *measures how x_i and x_j are important for predicting z_i*
- ▶ z_i gathers contextual information



A more complex Attention score: Keys, Queries and Values

- ▶ **Database Intuition:** it stores **(key,values)** that can be accessed with **queries**
- ▶ We associate for each token i :
 - ▶ A **query** vector $q(i)$, describing the information token i is interesting in/what I am looking for.
 - ▶ A **key** vector $k(i)$, whose goal is to match the relevant queries for token i /how to I communicate or align the information,
 - ▶ A **value** vector $v(i)$, describing the information contained by token i /what I hold.
- ▶ The attention score is defined as: α_{ij} as the dot product between $q(i)$ and $k(j)$.
- ▶ The vectors are obtained from trainable parameter matrices
→ the attention is assessed in a data-driven way.
 - ▶ W_Q for queries: $q(i) = W_Q \cdot x_i$
 - ▶ W_K for keys: $k(i) = W_K \cdot x_i$
 - ▶ W_V for values: $v(i) = W_V \cdot x_i$

Self-Attention Head: returns context vectors Z

- ▶ Rescaling:
 - ▶ softmax is sensitive to large numbers:
 $\text{softmax}([0.1, -0.3, -0.1, 0.4, 0.6])$ and
 $\text{softmax}([0.1, -0.3, -0.1, 0.4, 0.6])$
 - ▶ Consider Gaussian values $\sim N(0, 1)$, given 2 vectors u and v of dimension d
 - ▶ $\mathbb{E}[u \cdot v] = 0$ but $\text{Var}(u \cdot v) = d \Rightarrow$ increases with dimension d
 - ▶ But $\text{Var}(u \cdot v / \sqrt{d}) = 1$
- ▶ Then, $e_{ij} = q(i) \cdot k(j)$ and $\alpha_{ij} = \text{softmax}(e_{ij} / \sqrt{d})$
- ▶ $z_i = \sum_j \alpha_{ij} v(j)$

Self-Attention - General Matrix form

Let T be the size of the input, d the dimension of the embeddings, T' the size of the output, H the head size, O output dimension.

Input: $X \in \mathbb{R}^{T \times D}$, Output: $X' \in \mathbb{R}^{T' \times d'}$

We have the weight matrices $W_Q \in \mathbb{R}^{H \times d}$, $W_K \in \mathbb{R}^{H \times d'}$, $W_V \in \mathbb{R}^{O \times d'}$

$$\begin{cases} Q = W_Q \cdot X^T \in \mathbb{R}^{H \times T} \\ K = W_K \cdot X'^T \in \mathbb{R}^{H \times T'} \\ V = W_V \cdot X'^T \in \mathbb{R}^{O \times T'} \end{cases}$$

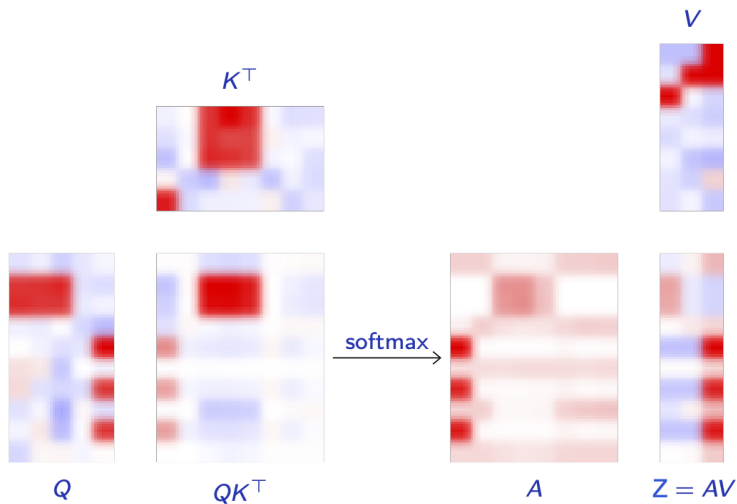
if $X = X'$ this is **self-attention** but could be more general.

W_Q, W_K, W_V are learned from data.

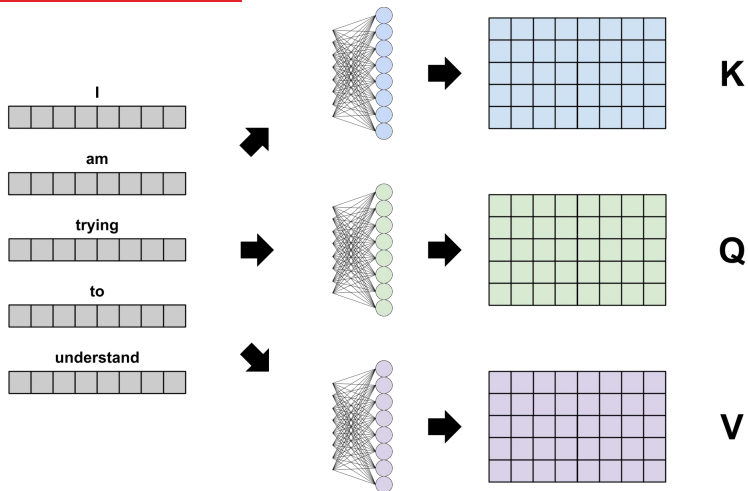
The matrices K and V have the same number of rows T' because each value is "indexed" by one key

$$Z = \underbrace{\text{softmax}(QK^T / \sqrt{d})}_\text{Attention matrix A} V$$

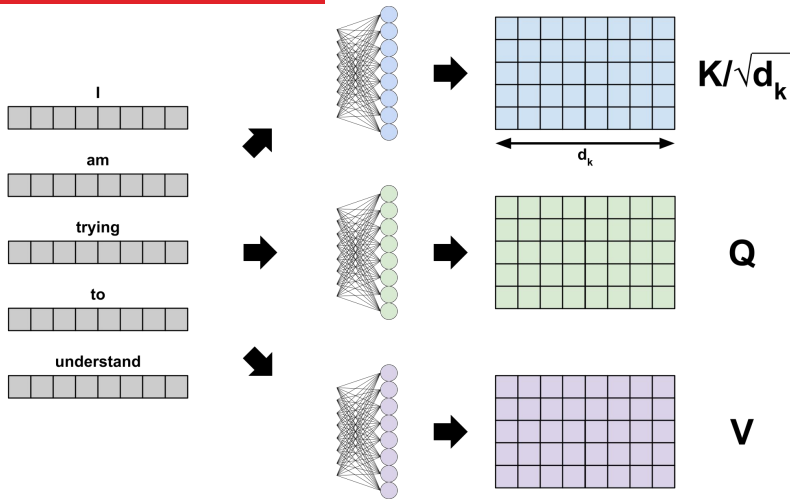
Attention Mechanism: Illustration with K , V , Q



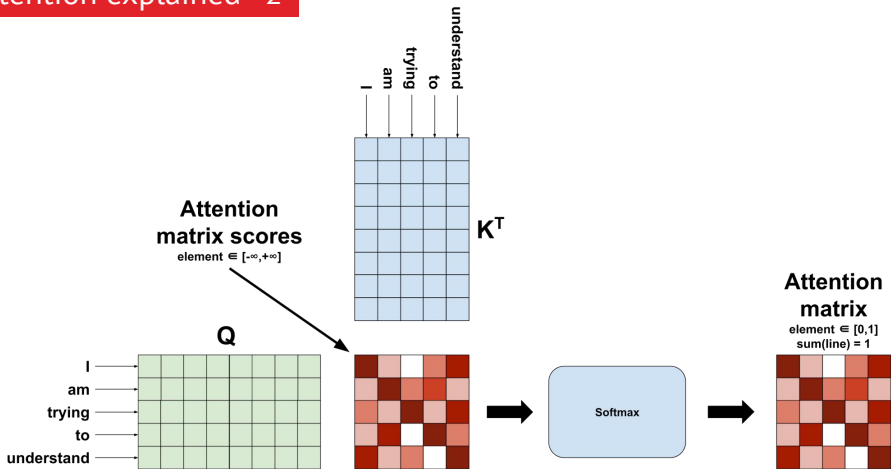
Attention explained - 1



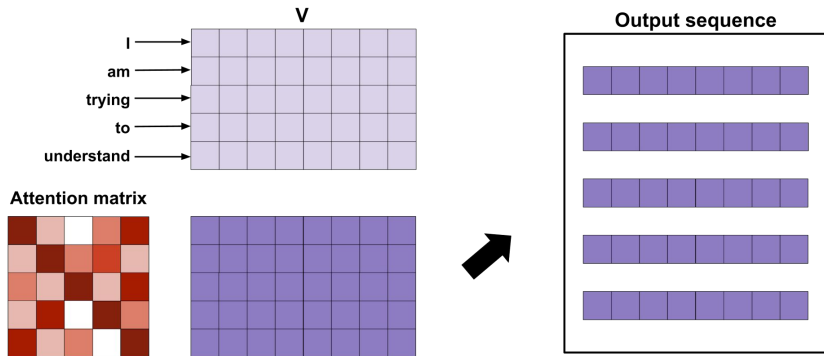
Attention explained - 1 (full)



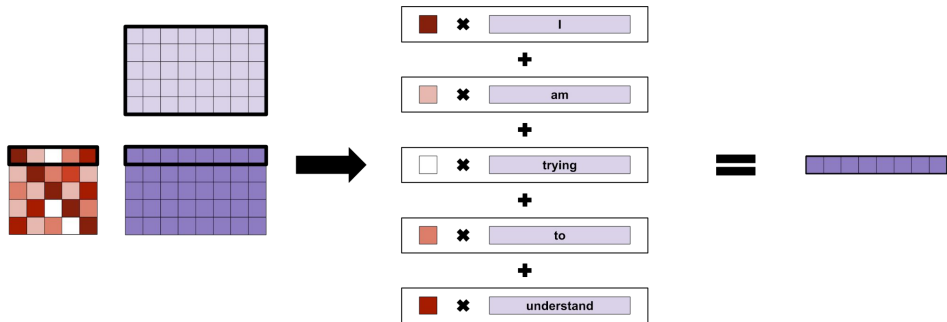
Attention explained - 2



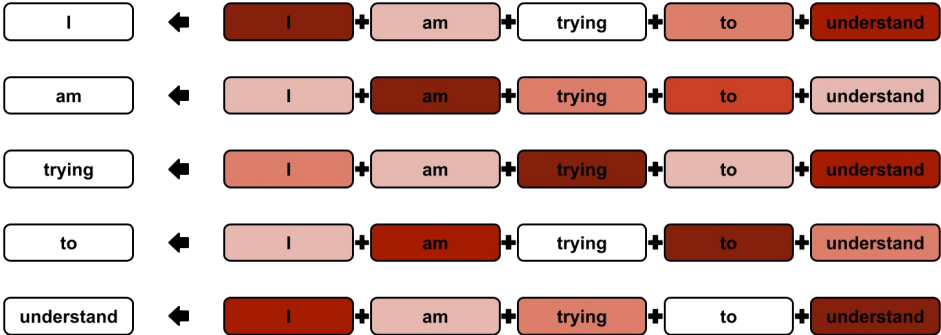
Attention explained - 3



Attention explained - 4



Intuition behind the attention mechanism - 0

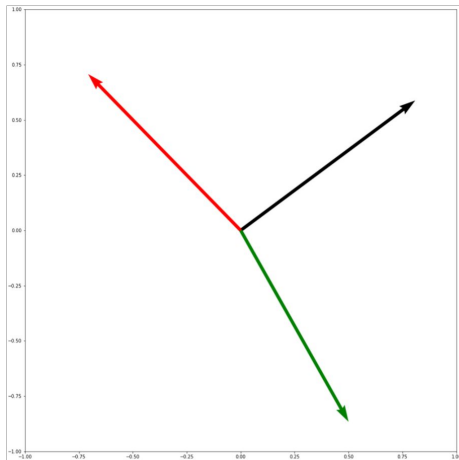


Intuition behind the attention mechanism - 1

The big dog



The : (0.50, -0.87)
big : (-0.70, 0.70)
dog : (0.81, 0.59)



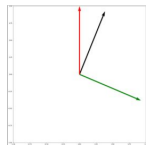
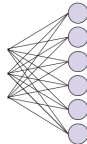
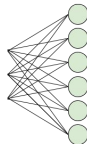
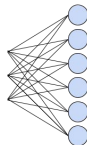
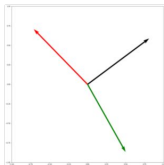
Intuition behind the attention mechanism - 2

0.50	-0.87
------	-------

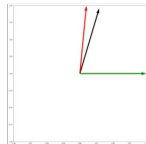
-0.70	0.70
-------	------

0.81	0.59
------	------

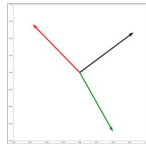
=



K

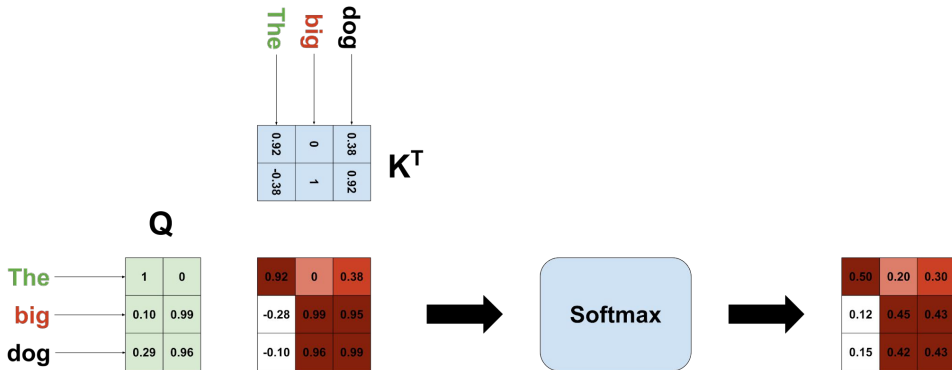


Q



V

Intuition behind the attention mechanism - 3



Intuition behind the attention mechanism - 4

0.50	0.20	0.30
0.12	0.45	0.43
0.15	0.42	0.43

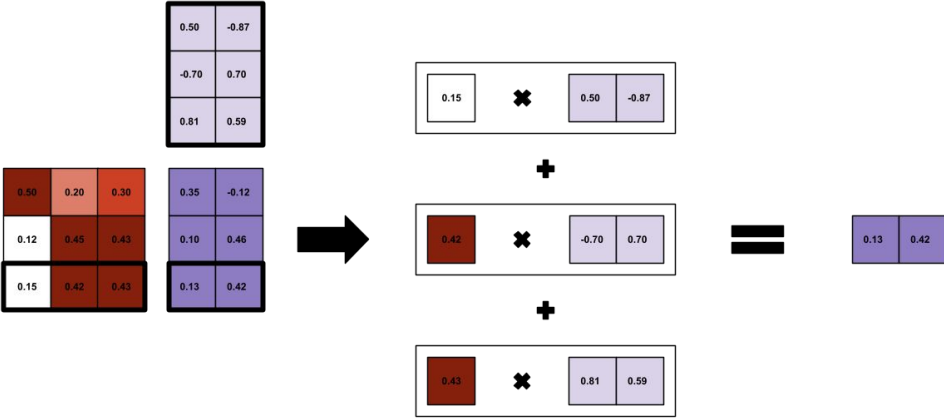


0.50	-0.87
-0.70	0.70
0.81	0.59

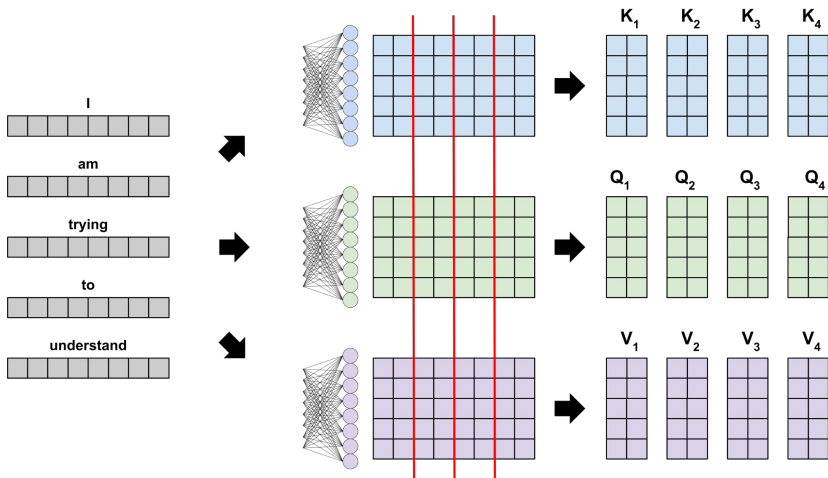


0.35	-0.12
0.10	0.46
0.13	0.42

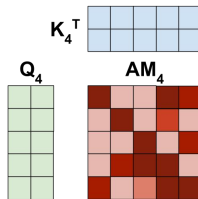
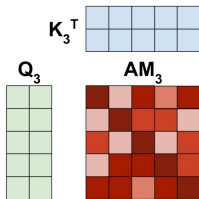
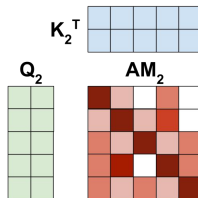
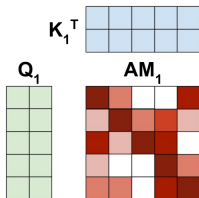
Intuition behind the attention mechanism - 4



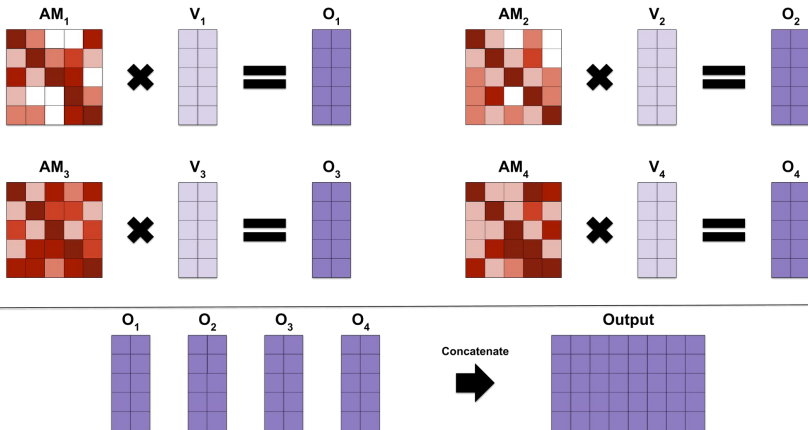
Multi-head Attention explained - 1



Multi-head Attention explained - 2



Multi-head Attention explained - 3



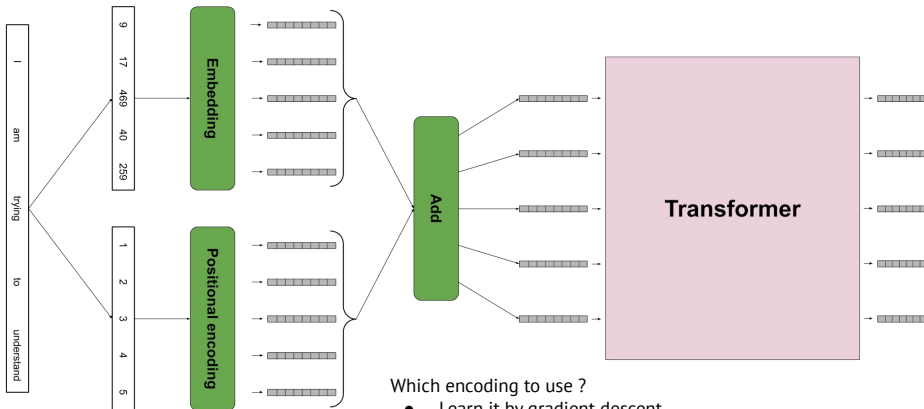
Positional embeddings

Keys, queries, values matrices include trainable parameters so that the attention mechanism learns where to put attention in a **data-driven way**.

But: The three matrices are the same for all indices! In other words, the attention mechanism is not aware of positions (neither absolute nor relative).

⇒ We need a way to encode the positions!

Positional encoding - 1



Which encoding to use ?

- Learn it by gradient descent
- Cosinus encoding

Positional encoding - 2

Decimal	Binary
0	0 0 0 0
1	0 0 0 1
2	0 0 1 0
3	0 0 1 1
4	0 1 0 0
5	0 1 0 1
6	0 1 1 0
7	0 1 1 1
8	1 0 0 0
9	1 0 0 1
10	1 0 1 0
11	1 0 1 1
12	1 1 0 0
13	1 1 0 1
14	1 1 1 0
15	1 1 1 1

Positional encoding - 3

Decimal	Binary
0	0 0 0 0
1	0 0 0 1
2	0 0 1 0
3	0 0 1 1
4	0 1 0 0
5	0 1 0 1
6	0 1 1 0
7	0 1 1 1
8	1 0 0 0
9	1 0 0 1
10	1 0 1 0
11	1 0 1 1
12	1 1 0 0
13	1 1 0 1
14	1 1 1 0
15	1 1 1 1

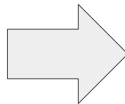
Inversion of bits every increment

Inversion of bits every 2 increments

Inversion of bits every 4 increments

Positional encoding - 4

Decimal	Binary
0	0 0 0 0
1	0 0 0 1
2	0 0 1 0
3	0 0 1 1
4	0 1 0 0
5	0 1 0 1
6	0 1 1 0
7	0 1 1 1
8	1 0 0 0
9	1 0 0 1
10	1 0 1 0
11	1 0 1 1
12	1 1 0 0
13	1 1 0 1
14	1 1 1 0
15	1 1 1 1



$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{model}})$$

$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{model}})$$

pos the position of the word in the sentence

i the dimension (index of a coordinate in the embedding)

d_{model} the total dimension of the embedding

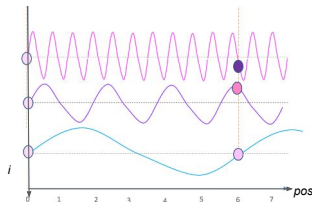
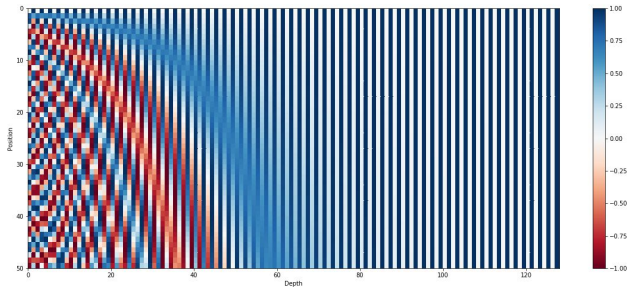


Figure from <https://www.youtube.com/watch?v=dichlcUZF0w>

Positional encoding - 5



$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{model}})$$

$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{model}})$$

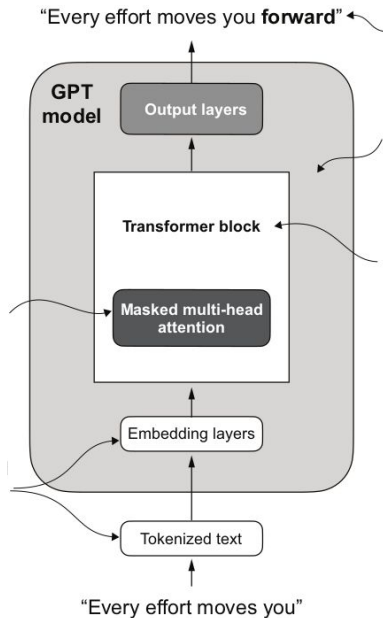
pos the position of the word in the sentence

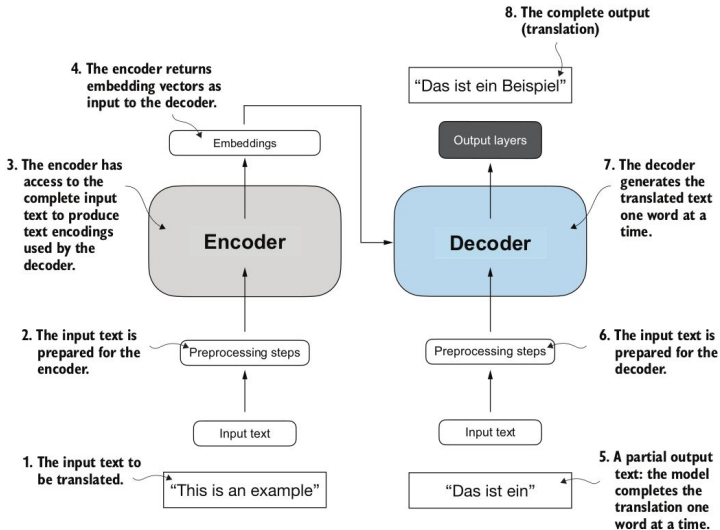
i the dimension (index of a coordinate in the embedding)

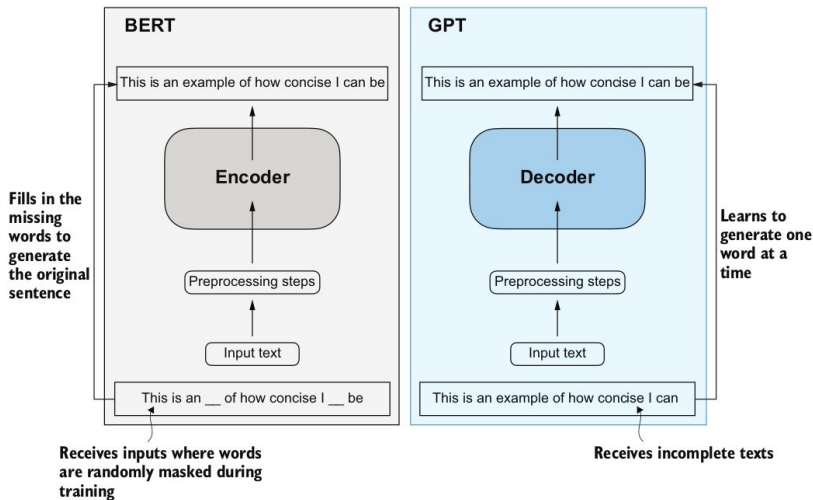
d_{model} the total dimension of the embedding

Figure from https://kazemnejad.com/blog/transformer_architecture_positional_encoding/

Principle of a Transformer and Architecture







AUTOREGRESSIVE MODELS

AUTOREGRESSIVE

It means that for generating a single new token we feed the model with the input + all tokens generated so far.

Creates the next word based on the input text

Iteration 1

"This is"

Output layers

Decoder

Preprocessing steps

Input text

"This"

Iteration 2

"This is an"

Output layers

Decoder

Preprocessing steps

Input text

"This is"

Iteration 3

"This is an example"

Output layers

Decoder

Preprocessing steps

Input text

"This is an"

The output of the previous round serves as input to the next round.

SLIDING WINDOWS



SLIDING WINDOWS

Text sample:

LLMs learn to predict one word at a time

LLMs learn to predict one word at a time

LLMs learn to **predict** one word at a time

LLMs learn to predict **one** word at a time

LLMs learn to predict one word at a time

LLMs learn to predict one word **at** a time

LLMs learn to predict one word at a time

LLMs learn to predict one word at a time

The LLM can't access words past the target.

Input the  LLM receives

Target to predict

DECODERS USE CAUSAL ATTENTION

	Your	journey	starts	with	one	step
Your	0.19	0.16	0.16	0.15	0.17	0.15
journey	0.20	0.16	0.16	0.14	0.16	0.14
starts	0.20	0.16	0.16	0.14	0.16	0.14
with	0.18	0.16	0.16	0.15	0.16	0.15
one	0.18	0.16	0.16	0.15	0.16	0.15
step	0.19	0.16	0.16	0.15	0.16	0.15

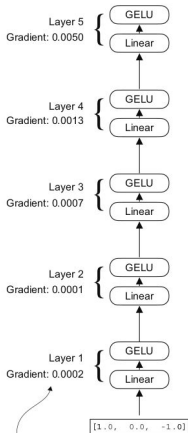
Attention weight for input tokens
corresponding to “step” and “Your”



	Your	journey	starts	with	one	step
Your	1.0					
journey	0.55	0.44				
starts	0.38	0.30	0.31			
with	0.27	0.24	0.24	0.23		
one	0.21	0.19	0.19	0.18	0.19	
step	0.19	0.16	0.16	0.15	0.16	0.15

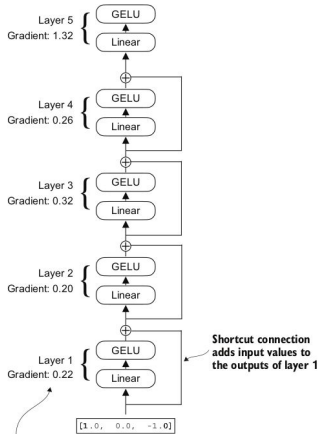
Masked out
future tokens
for the “Your”
token

Deep neural network

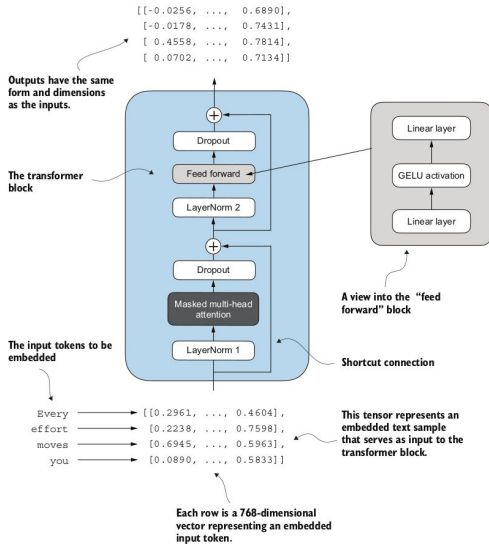


In very deep networks, the gradient values in early layers become vanishingly small

Deep neural network with shortcut connections



The shortcut connections help with maintaining relatively large gradient values even in early layers



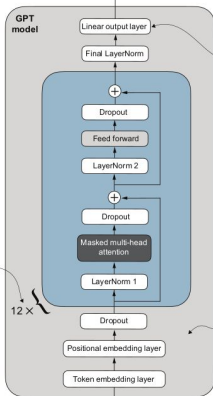
A $4 \times 50,257$ -dimensional tensor

```
[[-0.0055, ..., -0.4747],  
 [ 0.2663, ..., -0.4224],  
 [ 1.1146, ...,  0.0276],  
 [-0.8239, ..., -0.3993]]
```

The goal is for these embeddings to be converted back into text such that the last row represents the word the model is supposed to generate (here, the word "forward").

The transformer block is repeated 12 times.

12 ×



The last linear layer embeds each token vector into a 50,257-dimensional embedding, where 50,257 is the size of the vocabulary.

The GPT code implementation includes a token embedding and positional embedding layer (see chapter 2).

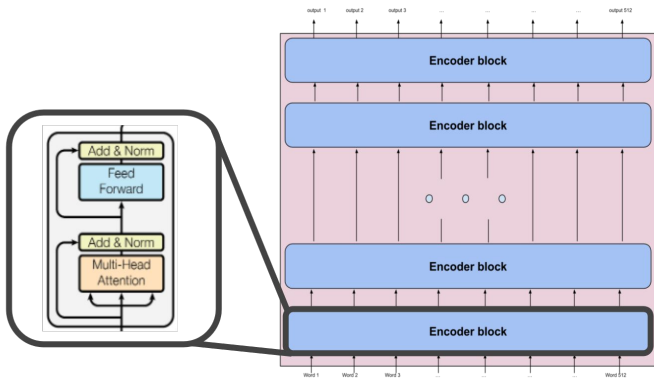
Every effort moves you

Transformer architectures

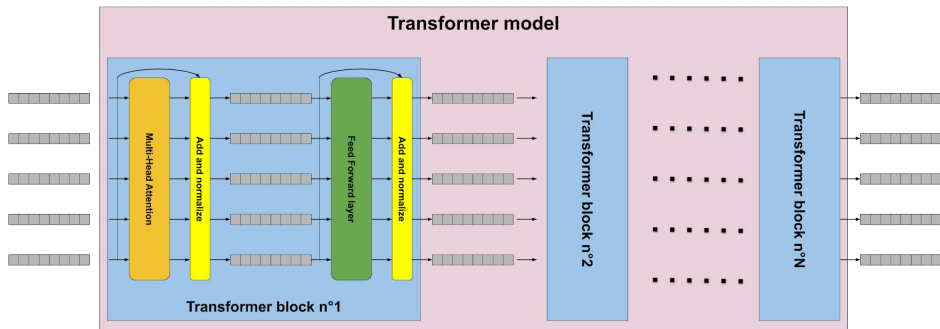
The most common transformer architectures

- Encoder (BERT)
- Decoder (GPT)
- Encoder-Decoder

Auto-encoding model (BERT or Encoder)

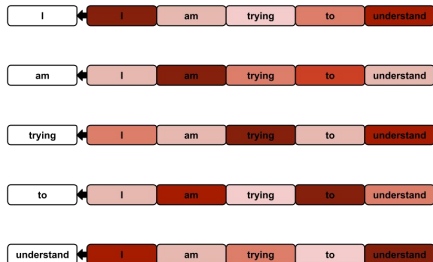


Auto-encoding model (BERT or Encoder)



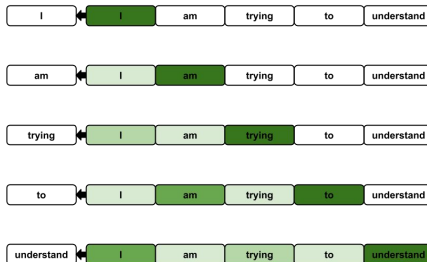
Bidirectional vs Unidirectional attention

Bidirectional attention (BERT - Encoder - Auto-encoding)

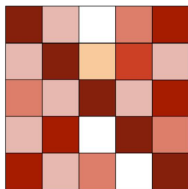


VS

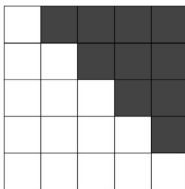
Unidirectional attention (GPT - Decoder - Auto-regressive)



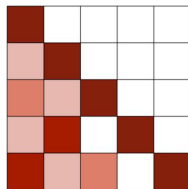
Bidirectional vs Unidirectional attention



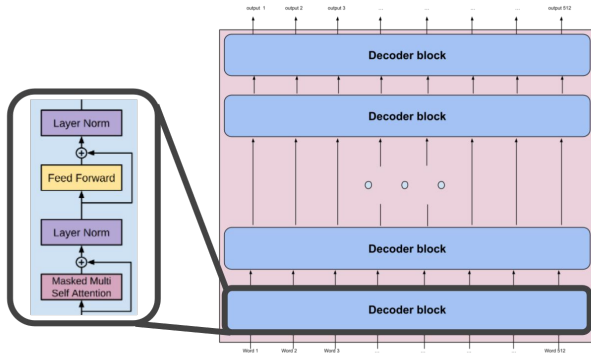
−



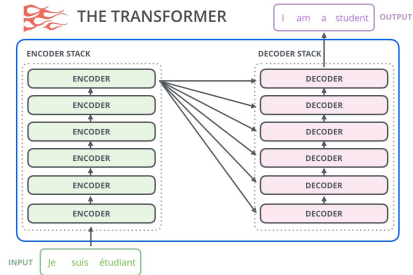
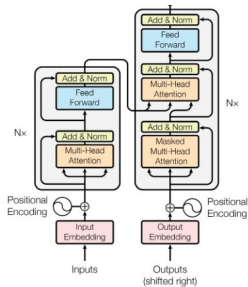
=



Auto-regressive model (GPT or Decoder)



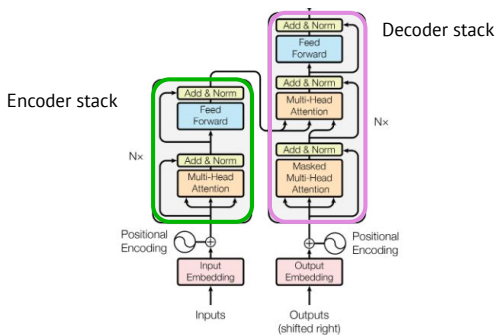
Encoder-Decoder model (1st



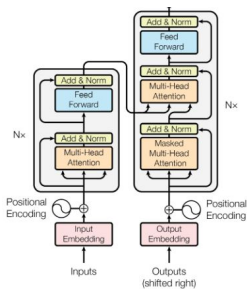
Attention is all you need, Vaswani A. et al, 2017

<https://jalamar.github.io/illustrated-gpt2/>

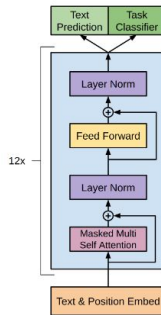
Small trap: misuse of language - 1



Small trap: misuse of language - 2



Attention is all you need, Vaswani A. et al, 2017



Improving Language Understanding by Generative Pre-Training, Radford et al

Pretraining

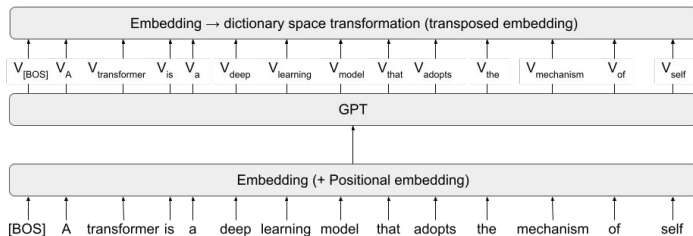
NLP covers multiple tasks:

- Question answering
- Inference
- Paraphrasing
- Grammar (is a given sentence correct grammatically?)
- Sentiment analysis
- etc

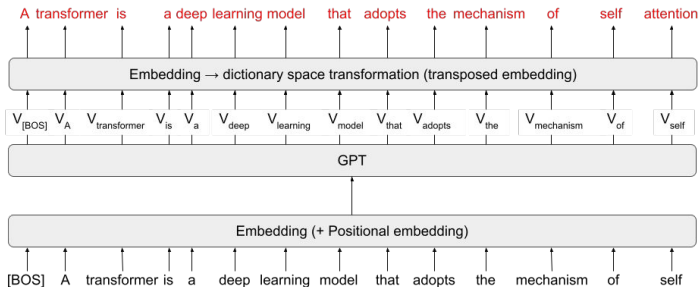
But the language is always the same

Can we pool trainings (at least partially)?

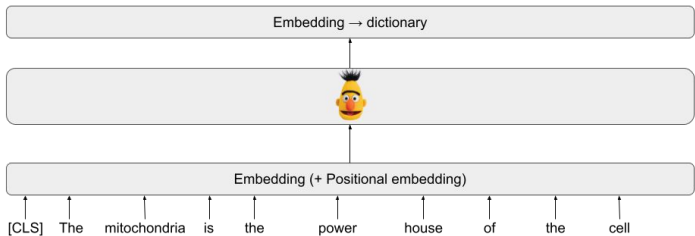
GPT pretraining



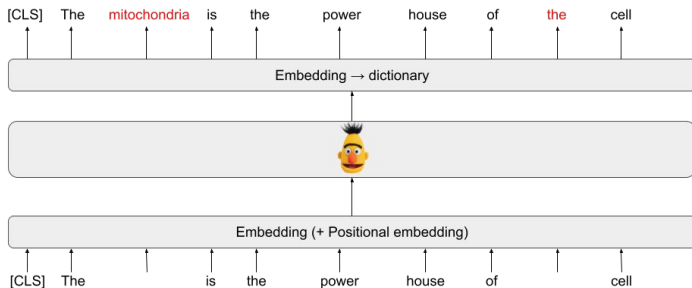
GPT pretraining: predict the next word



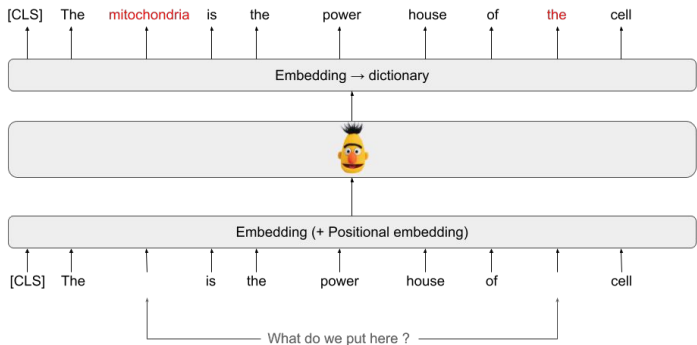
BERT pretraining



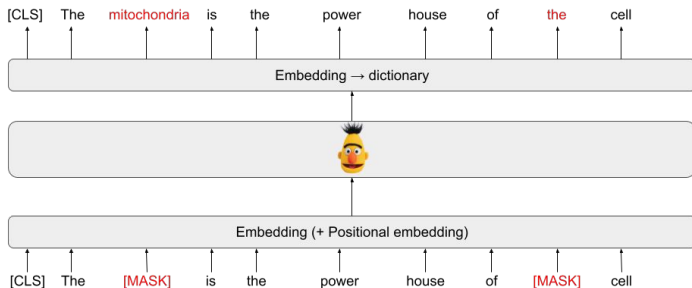
BERT pretraining: Masked Language Model 1



BERT pretraining: Masked Language Model 2

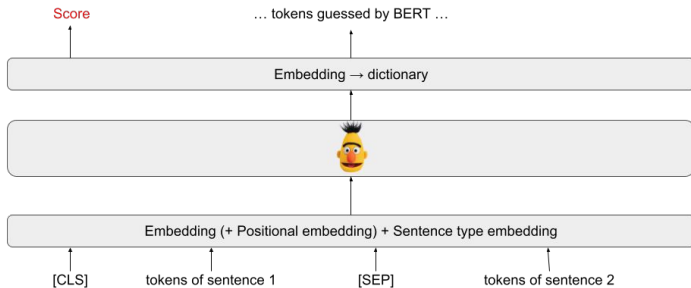


BERT pretraining: Masked Language Model 3



BERT pretraining: Next Sentence Prediction

Classification score : does sentence 2 comes right after sentence 1 ?



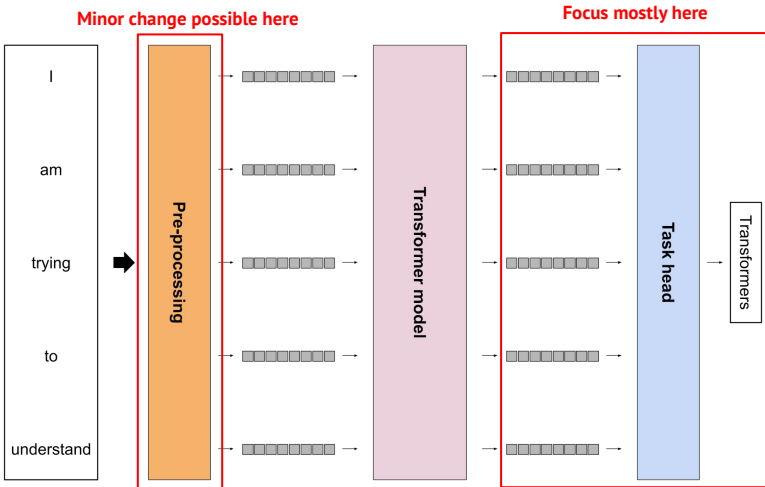
BERT pretraining

- Next Sentence Prediction is actually detrimental

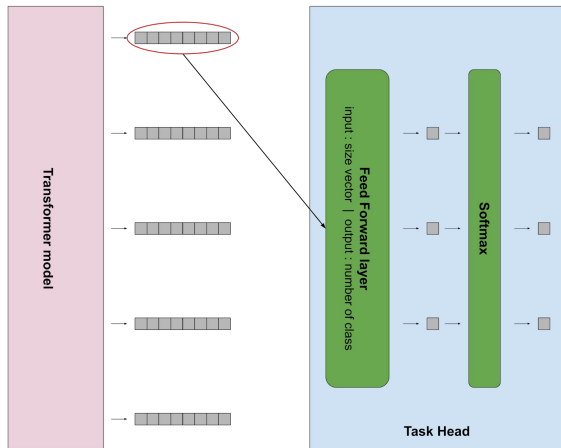
Source: RoBERTa: A Robustly Optimized BERT Pretraining Approach, Liu Y. et al, 2019, <https://arxiv.org/pdf/1907.11692.pdf>

- You can use multiple objectives (as BERT did, or other variants like SpanBERT)

Classical fine tuning



Example of a classical fine tuning - classification



Text classification dataset

Input: reviews (text)

Label: 0, 1 or 2 (negative, neutral, positive)

Prompting example

Dataset

Input: **Reviews**

Label: **Positive, neutral or negative**

Template

The comment is



Prompting

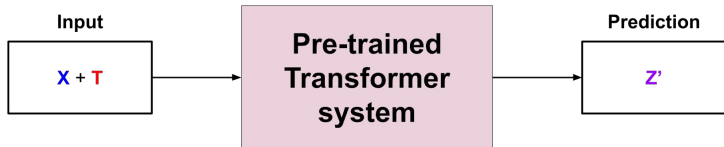
Dataset

Input: **X** (text)

Label: **Z** (can be formulate as text)

Template

T (text)



Exemple de prompting

Type	Task	Input ([X])	Template	Answer ([Z])
Text CLS	Sentiment	I love this movie.	[X] The movie is [Z].	great fantastic ...
	Topics	He prompted the LM.	[X] The text is about [Z].	sports science ...
	Intention	What is taxi fare to Denver?	[X] The question is about [Z].	quantity city ...
Text-span CLS	Aspect Sentiment	Poor service but good food.	[X] What about service? [Z].	Bad Terrible ...
Text-pair CLS	NLI	[X1]: An old man with ... [X2]: A man walks ...	[X1]? [Z], [X2]	Yes No ...
Tagging	NER	[X1]: Mike went to Paris. [X2]: Paris	[X1] [X2] is a [Z] entity.	organization location ...
Text Generation	Summarization	Las Vegas police ...	[X] TL;DR: [Z]	The victim ... A woman
	Translation	Je vous aime.	French: [X] English: [Z]	I love you. I fancy you. ...

Pre-train, Prompt, and Predict: A Systematic Survey of Prompting Methods in Natural Language Processing, Pengfei Liu and al

Transformers with images

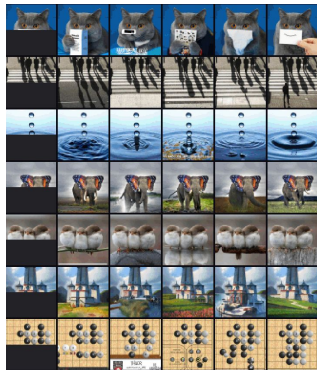
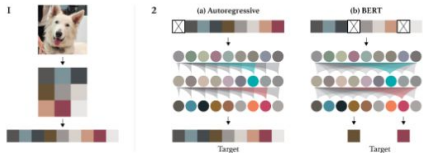
Not only for words but any 1D input.

What about 2D inputs (for instance pictures) ?

- ImageGPT
- Visual Transformer

Also useful in other fields: chemistry, biology...

ImageGPT

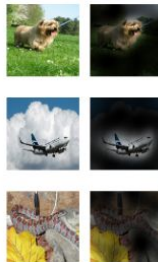
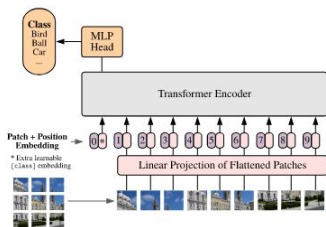


Generative Pretraining from Pixels, Chen M. et al, 2020

https://cdn.openai.com/papers/Generative_Pretraining_from_Pixels_V2.pdf

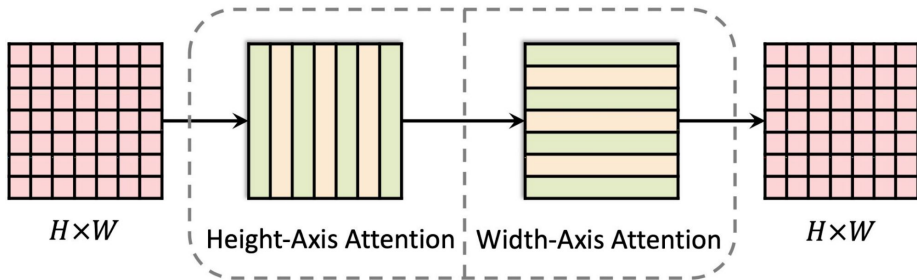
<https://openai.com/blog/image-gpt/>

Visual Transformer



An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale, Dosovitskiy A et al, 2021: <https://arxiv.org/pdf/2010.11929.pdf>

Axial Attention



Bibliography

Papers:

- Attention Is All You Need (<https://arxiv.org/abs/1706.03762>)
- BERT (<https://arxiv.org/abs/1810.04805>)
- GPT (https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf)
- GPT3 (<https://arxiv.org/abs/2005.14165>)
- T5 (<https://arxiv.org/abs/1910.10683>)
- BLOOM (<https://arxiv.org/abs/2211.05100>)
- Foundation Models (<https://arxiv.org/abs/2108.07258>)

Book:

- Natural Language Processing with Transformers

Illustration

<https://jalammar.github.io/illustrated-transformer/>

Conclusion

Conclusion

- ▶ Many NLP models are based on Transformer architectures. Computer Vision starts to have its own models (not seen here)
- ▶ Importance of Attention: self-attention on the same sequence as input, cross-attention on a different sequence (e.g. for translation)
- ▶ As you already saw, finding the architecture for a problem is tricky → Better to start from an existing model.
- ▶ Very (too?) active field: lots of existing resources (papers, models, implementations, frameworks, ...)
- ▶ Large Language Models (LLM) can be built only from few big institutions
- ▶ New perspectives for many applications.